

Assignment 1: Vending Machine

Due Date

- **Board Implementation – March 24(Mon) & March 26(Wed) In-Class**
- **Report – March 28 (Fri), 11:59 PM**

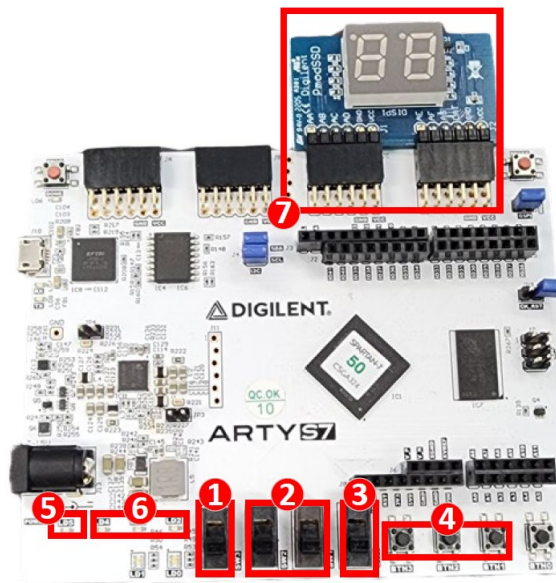
Notice

- Ensure that there are no Verilog HDL syntax errors.
- The score for the 1st Assignment will be based on the Board Implementation and the Report.
- Design the Vending Machine using the provided skeleton code.
- The project code must be implemented using a **“Finite State Machine”**.
- **DO NOT** modify the I/O ports of the top module without modifying XDC file, as it may cause errors during implementation.
- **Generate the bitstream file in Vivado** before attending the board implementation classes.

In the first assignment, we will design a Vending Machine module. The provided skeleton code defines the required modules and I/O. Write the code to implement the specified operations for board implementation. Modifying the I/O ports of the top module without altering the board configuration is not recommended, as it may cause implementation errors.

The vending machine sells three items at different prices: **3, 5, and 7**. Before selling, the stock of each item must be manually refilled. If an item is out of stock, the vending machine will indicate this by lighting the LEDs. To purchase an item, users can insert three types of coins (**1, 5, and 10**) until the balance matches the item's price, then select the item. Once an item is purchased, the balance will decrease by the item's price, and the stock of the selected item will decrease by 1. **The maximum stock for each item is 5, and the maximum balance is 50.** To implement the vending machine's operation on the FPGA board, the process is divided into three modes: **Item Filling, Coin Inserting, Selling**. These modes will be controlled using switches on the FPGA board.

The figure below illustrates the FPGA board setup. The design will utilize: 3 switches (SWs), 4 buttons (BTNs), 4 LEDs, Seven-segment displays (SSDs).



1. **SW3**: Turns the vending machine **ON/OFF**. Other parts of the FPGA board will function **only when SW3 is HIGH**.
2. **SW1~SW2**: **Control the mode** of the vending machine. The truth table below defines the selected mode based on **SW1 and SW2**.

Modes		SW2	
		0	1
SW1	0	Selling	Item Filling
	1	Coin Inserting	Coin Inserting

3. **SW0**: **Resets** the vending machine. When switched **HIGH**, the vending machine stops operating. After switching it **LOW**, both **stock and balance** are reset to **0**.
4. **BTN1~BTN3**: Used to perform the tasks according to the vending machine's mode: adds stock, inserts coins, selects an item.

	Item Filling	Coin Inserting	Selling
BTN1	Fill Item 1 (Price: 3)	Insert Coin 1 (Value: 1)	Select Item 1
BTN2	Fill Item 2 (Price: 5)	Insert Coin 2 (Value: 5)	Select Item 2
BTN3	Fill Item 3 (Price: 7)	Insert Coin 3 (Value: 10)	Select Item 3

5. **LED5**: Turns **HIGH** while the vending machine is operating.
6. **LED2–LED4**: Indicate whether an item is **out of stock** or **available**, depending on the mode. LEDs turn **HIGH** only under the conditions specified in the table below.

<i>ON</i>	Item Filling	Selling
LED2	Item 1 is most recently filled.	Item 1 is out of stock.
LED3	Item 2 is most recently filled.	Item 2 is out of stock.
LED4	Item 3 is most recently filled.	Item 3 is out of stock.

7. **SSD**: Displays the **stock** of the most recently filled item in **Item Filling mode** and the **balance** in **Selling and Coin Inserting modes**.

The vending machine must operate at **50MHz**, while the default clock signal generated by the FPGA is **100MHz**. To convert the clock signal, you can use the **clock divider** from the **Week 2 Practice**. Additionally, the buttons on the FPGA are highly susceptible to noise. To ensure stable operation by eliminating noise, use **bouncer** from the **Week 2 Practice**. Other design approaches are allowed, but ensure that the module functions correctly.

Along with the announcement, a skeleton code with defined input/output ports and XDC files will be uploaded. XDC files for both the Arty S7-50 and Arty A7-35 boards will be provided. Please use the XDC file corresponding to the board you used in the Week 3 class to generate the bitstream file and participate in the board implementation session. (If you generate bitstreams for both boards, you may be able to reduce waiting time.)

Here are some tips for coding in Verilog:

1. **Avoid assigning multiple *reg* variables in the same *always* loop.** The compiler may synthesize latches for undefined cases, which can lead to errors. For beginners, it is highly recommended to use **a separate *always* loop for each *reg* variable**.
2. **Ensure all variable cases are defined.**
 - Every *if* and *else if* condition should be **followed by an *else***.
 - Every *case* statement should include a ***default*** case.
3. **Do not mix blocking (=) and non-blocking (<=) assignments in the same *always* loop or for the same variable.**